

Projet fil rouge : “Mini Bibliothèque” (Catalogue de livres)

Données visées (à terme)

Un livre aura :

- `id`
 - `title`
 - `author`
 - `year`
 - `genre`
 - `price`
 - `in_stock` (bool)
 - `created_at`
-

Phase 1 — Exercices de bases PHP (sans formulaire, sans DB)

Exercice 1 — Types, concaténation, affichage HTML

Objectif : comprendre variables + types + affichage propre.

- Déclare des variables : `string`, `int`, `float`, `bool`.
 - Affiche une “fiche livre” en HTML (titre, auteur, année, prix, en stock oui/non).
 - Bonus : arrondir le prix à 2 décimales (`number_format`).
-

Exercice 2 — Conditions complètes + opérateurs

Objectif : `if` / `elseif` / `else`, opérateurs logiques.

- À partir de `year`, afficher “Classique” si `< 1980`, “Moderne” si `1980-2010`, “Récent” si `> 2010`.
 - À partir de `price` :
 - `< 10` → “Bon marché”
 - `10-25` → “Standard”
 - `> 25` → “Premium”
 - Bonus : afficher un badge “🔥 En promo” si `in_stock == true` ET `price < 12`.
-

Exercice 3 — Fonctions (retour, paramètres, valeurs par défaut)

Objectif : écrire des fonctions réutilisables.

- Crée `formatPrice(float $price): string`
 - Crée `formatStock(bool $inStock): string` qui retourne `"In stock"` ou `"Out of stock"`
 - Crée `getBookLabel(int $year, float $price): string` (combine les règles des exercices précédents)
 - Bonus : valeur par défaut sur un paramètre (`$currency = "CHF"`).
-

Exercice 4 — Tableaux indexés + boucles + HTML

Objectif : `foreach`, structurer une liste.

- Crée un tableau `$books` (indexé) avec **5 titres** (juste des strings).
 - Affiche-les en `<ul>`.
 - Bonus : afficher un index (`#1`, `#2`, ...) avec une boucle.
-

Exercice 5 — Tableaux associatifs (données réalistes)

Objectif : structure type "enregistrement".

- Un livre = tableau associatif :
 - `title`, `author`, `year`, `genre`, `price`, `in_stock`
 - Crée **3 livres** et affiche-les en HTML.
 - Bonus : afficher un message si une clé manque (`isset`).
-

Exercice 6 — Tableau de tableaux (mini "base de données" en mémoire)

Objectif : manipuler une "collection".

- Crée `$library` = tableau contenant **au moins 6 livres associatifs**.
 - Affiche un tableau HTML `<table>` avec colonnes (Title, Author, Year, Genre, Price, Stock).
 - Bonus : surligner les livres "Out of stock" (classe CSS).
-

Exercice 7 — Tri + filtres manuels (sans fonctions PHP avancées au début)

Objectif : logique + boucles.

- Affiche uniquement les livres `in_stock = true`.

- Affiche uniquement les livres d'un genre donné (ex: "Fantasy").
 - Bonus : compter combien de livres sont en stock et afficher le total.
-

Exercice 8 — Tri (niveau +)

Objectif : introduire `usort` + comparaison.

- Trier `$library` par `year` décroissant.
 - Trier `$library` par `price` croissant.
 - Bonus : tri multi-critères (genre puis title).
-

Exercice 9 — Fonctions + validation de données

Objectif : vérifier proprement les entrées / données.

- Écrire `validateBook(array $book): array` qui renvoie une liste d'erreurs.
 - `title` min 2 caractères
 - `year` entre 1500 et année actuelle
 - `price` `>= 0`
 - Bonus : refuser les genres non autorisés (liste blanche).
-

Exercice 10 — Lecture/écriture fichiers (JSON)

Objectif : persistance sans DB.

- Sauvegarder `$library` dans `library.json` (`json_encode`, `file_put_contents`).
 - Lire `library.json` et reconstruire `$library` (`file_get_contents`, `json_decode`).
 - Bonus : si le fichier n'existe pas, créer une bibliothèque vide.
-

Phase 2 — Web : formulaires, GET/POST, sessions

Exercice 11 — Formulaire POST : ajouter un livre (sans DB)

Objectif : récupérer `$_POST`.

- Formulaire `add.php` (title, author, year, genre, price, in_stock).
 - À la soumission : valider + afficher un résumé propre.
 - Bonus : `htmlspecialchars` partout à l'affichage.
-

Exercice 12 — Sessions : stocker la bibliothèque

Objectif : simuler une “app”.

- Stocker `$library` dans `$_SESSION`.
 - Page `index.php` : liste des livres.
 - Page `add.php` : ajoute un livre à la session puis redirige vers `index.php`.
 - Bonus : message “Book added successfully” en session (flash message).
-

Exercice 13 — GET : filtrer et rechercher

Objectif : `$_GET` et paramètres.

- Ajouter un champ `q` (recherche dans title ou author).
 - Ajouter un filtre `genre`.
 - Bonus : si aucun résultat, afficher un message.
-

Exercice 14 — Mini “CRUD” en session

Objectif : comprendre CRUD avant DB.

- Ajouter un `id` unique (généré) à chaque livre.
 - Create : add
 - Read : list + view `book.php?id=...`
 - Update : `edit.php?id=...` modifie un livre en session
 - Delete : `delete.php?id=...`
 - Bonus : refuser si `id` inexistant.
-

Phase 3 — Base de données (PDO) + CRUD final

Exercice 15 — Modélisation DB + script SQL

Objectif : créer la table.

Table `books` :

- `id` INT AI PK
- `title` VARCHAR(150) NOT NULL
- `author` VARCHAR(120) NOT NULL
- `year` INT NOT NULL
- `genre` VARCHAR(80) NOT NULL

- `price` DECIMAL(10,2) NOT NULL
- `in_stock` TINYINT(1) NOT NULL DEFAULT 1
- `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP

Bonus : index sur `author` et `genre`.

Exercice 16 — Connexion PDO + fichier `db.php`

Objectif : connexion propre et réutilisable.

- Créer `db.php` qui retourne `PDO`.
 - Activer exceptions + fetch assoc.
 - Bonus : config `.env` simple (ou constants) sans hardcoder partout.
-

Exercice 17 — READ : afficher tous les livres depuis DB

Objectif : SELECT + affichage.

- `index.php` lit `books` et affiche table.
 - Bonus : tri (ORDER BY `created_at` DESC).
-

Exercice 18 — CREATE : ajouter un livre en DB

Objectif : INSERT + requêtes préparées.

- Formulaire `add.php`
 - Insertion préparée
 - Redirection vers index
 - Bonus : réafficher les erreurs sans perdre les champs.
-

Exercice 19 — UPDATE : modifier un livre

Objectif : SELECT by id + UPDATE.

- `edit.php?id=...` charge le livre
 - formulaire pré-rempli
 - UPDATE préparé
 - Bonus : si id invalide → "Book not found" + code HTTP 404 (simple).
-

Exercice 20 — DELETE : supprimer un livre

Objectif : suppression sécurisée.

- `delete.php?id=...` (POST recommandé)

- Confirmation côté UI
 - Bonus : empêcher suppression via GET (accepter seulement POST).
-

Exercice 21 — Recherche DB + filtres (niveau +)

Objectif : requêtes dynamiques sécurisées.

- Recherche `q` (title OR author)
 - Filtre genre
 - Filtre stock (in_stock)
 - Bonus : pagination (LIMIT / OFFSET).
-

Exercice 22 — Finalisation : mini architecture propre

Objectif : lisibilité + maintenance.

- `src/BookRepository.php` :
 - `findAll`, `findById`, `create`, `update`, `delete`, `search`
 - `public/` pour les pages
 - Bonus : page `seed.php` qui insère 10 livres d'exemple.
-

Livrable final (CRUD fonctionnel DB)

- `public/index.php` (liste + filtres)
- `public/add.php` (create)
- `public/book.php?id=` (read détail)
- `public/edit.php?id=` (update)
- `public/delete.php` (delete POST)
- `src/BookRepository.php`
- `db.php`
- `books.sql`